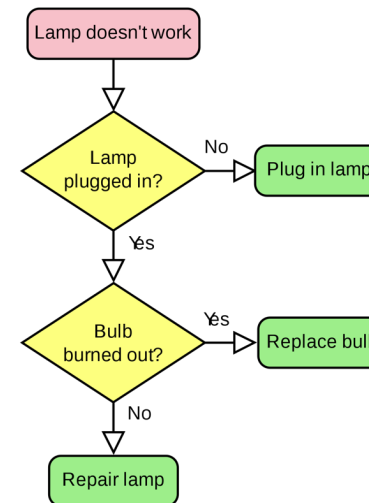
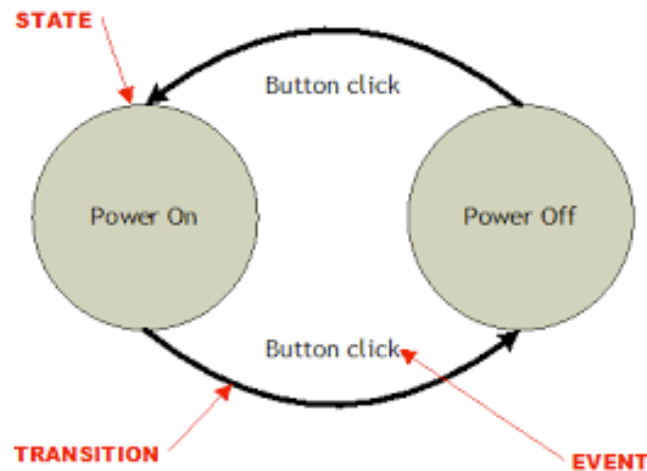


C03053 – Embedded Systems

4. State Machine & Flow-Chart



Learning Outcome

- Students are expected to be able to ...

Describe an embedded system using state machine/chart/diagram

Represent algorithms in embedded softwares using flow-chart



State Machine Components in Depth

- **State**

A distinct condition or mode the system can be in at a given time (e.g., Opened, Closed).

- **Transition**

A directed change from one state to another, triggered by an event.

- **Event**

Something that happens — a button press, a sensor reading, a timer expiring — that can trigger a transition.

- **Guard condition**

A boolean condition that must be true for a transition to fire, even if the triggering event occurs (e.g., $\text{Temp} > 50$).

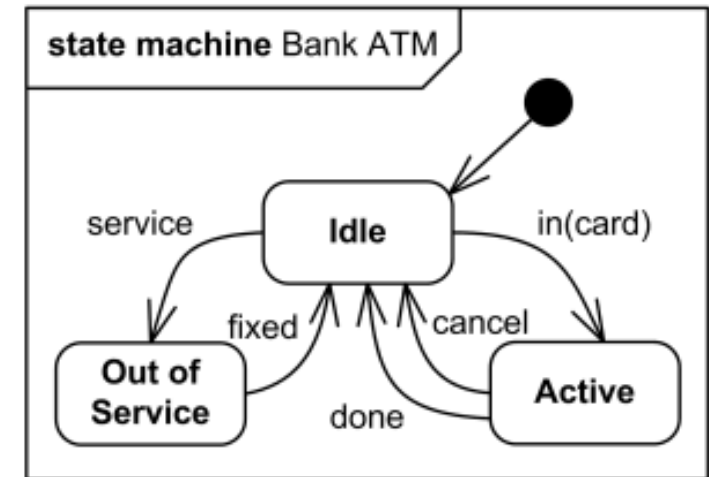
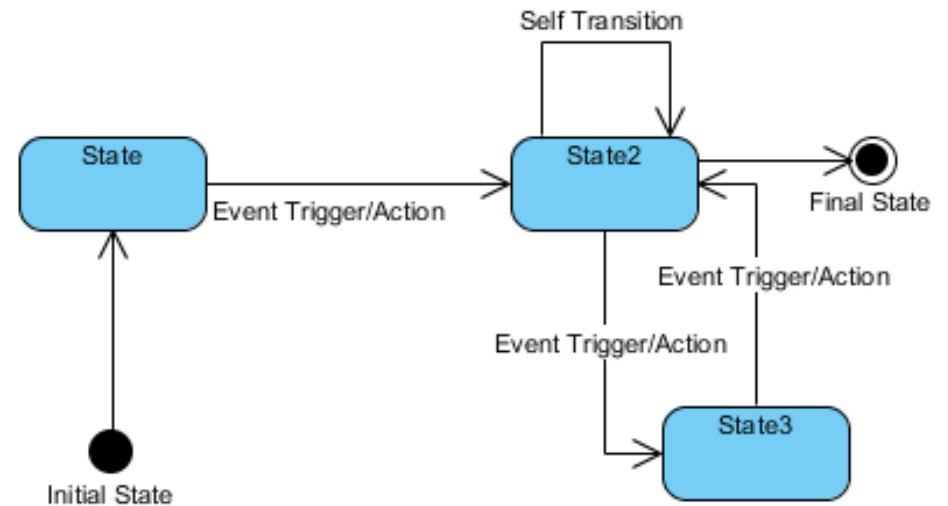
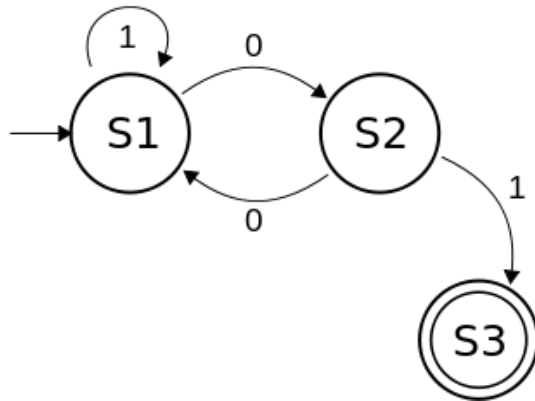
- **Action / Output**

Something the system does — turning on an LED, starting a timer — as a result of being in a state or of a transition firing.



State Machine Diagram

- A state machine is any device that stores **the status** of something at a given time and can operate on **input** to change the status and/or cause an action or **output** to take place for any given change.

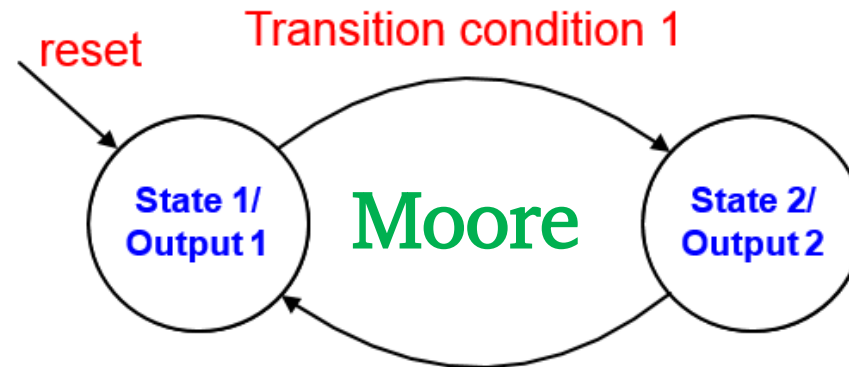


System Description by State Machine

■ State

Present state

Next state



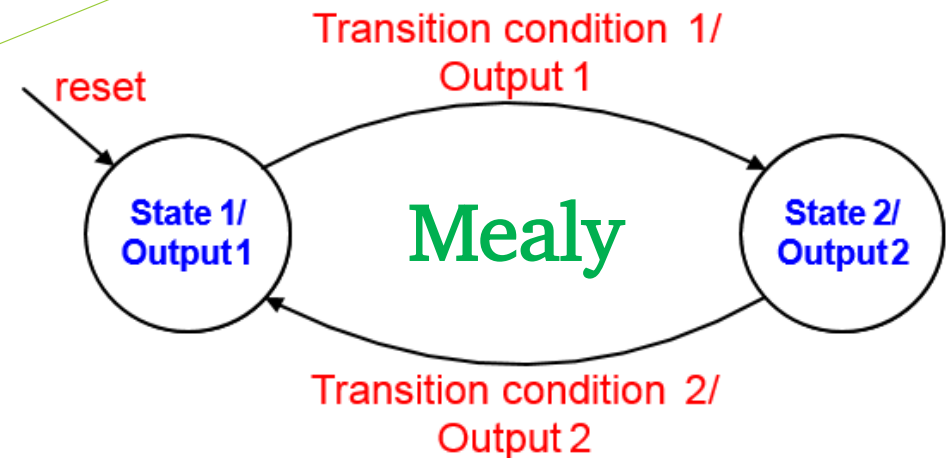
■ Input

Transition condition

Event

■ Output

Functions of the present states (inputs)



Moore vs. Mealy — A Closer Comparison

- **Moore machine**

Output depends only on the current state.

Output is stable for as long as the system stays in that state — easy to reason about, less prone to glitches.

Example: the traffic light — each state (NSGo, NSToStop, ...) has one fixed LED output.

- **Mealy machine**

Output depends on both the current state AND the current input.

Can react faster (output can change the instant an input changes, without waiting for a state transition) but is more prone to short output glitches if inputs are noisy.

Often produces fewer states for the same behavior, since one state can produce different outputs for different inputs.

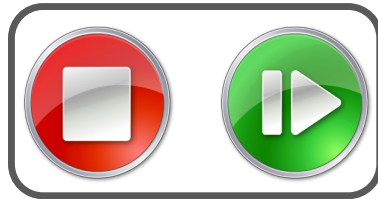
- **Rule of thumb: prefer Moore for outputs that must be clean and stable (like our LEDs); prefer Mealy when you need the fastest possible reaction to an input change.**



Example 1

- Music Player

stop button
start/pause

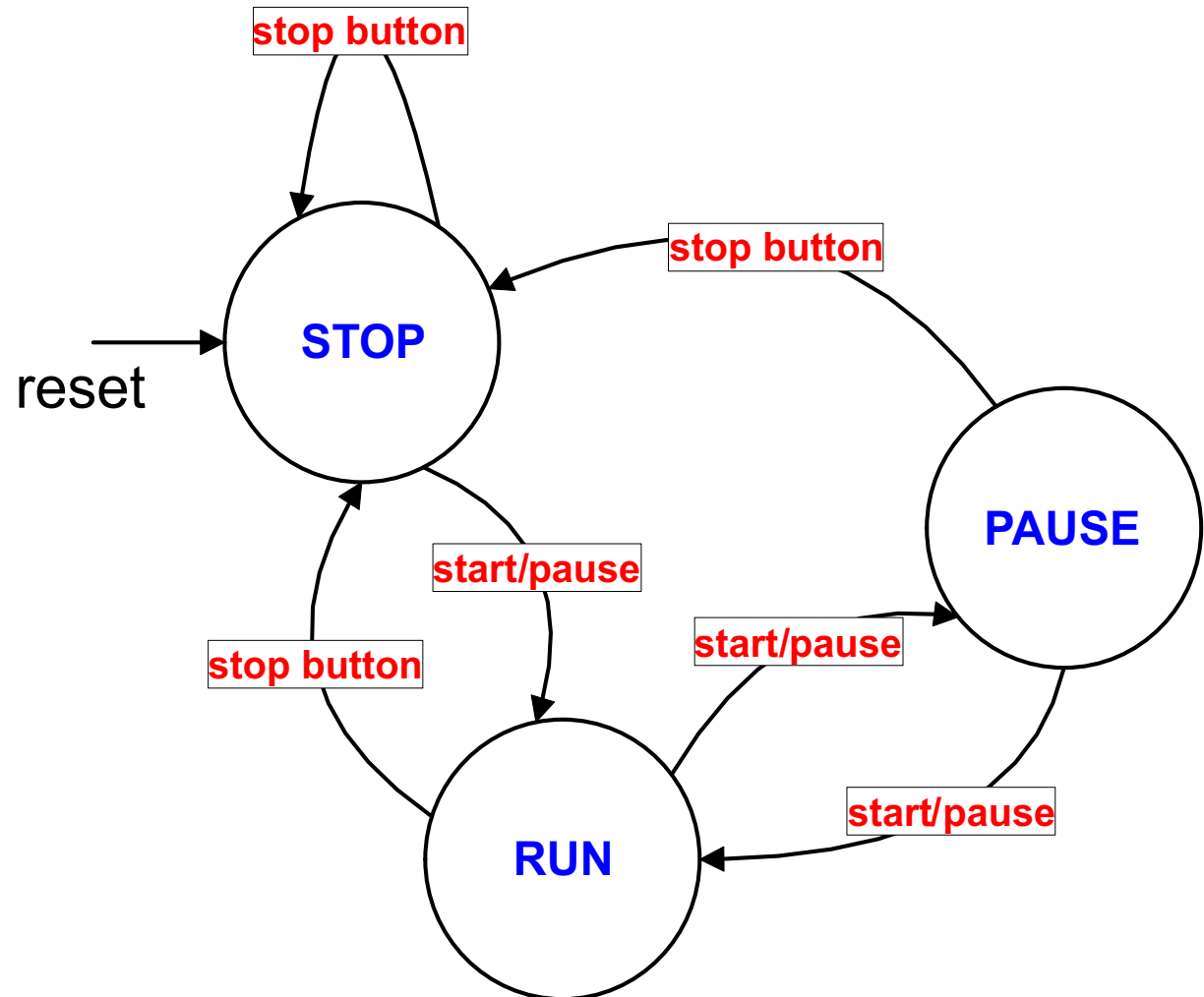


- State determination

STOP
RUN
PAUSE

- Transition condition

Button click



Example 2

- **Garage Door Control**

Remote control

Door Actuators

- **State determination**

- **Transition condition**



Example 2

- **Garage Door Control**

Remote control
Door Actuators



- **State determination**

Opened
Closed

- **Transition condition**

Button click (btnUp/btnDown)



Example 2

■ Garage Door Control

Remote control
Door Actuators

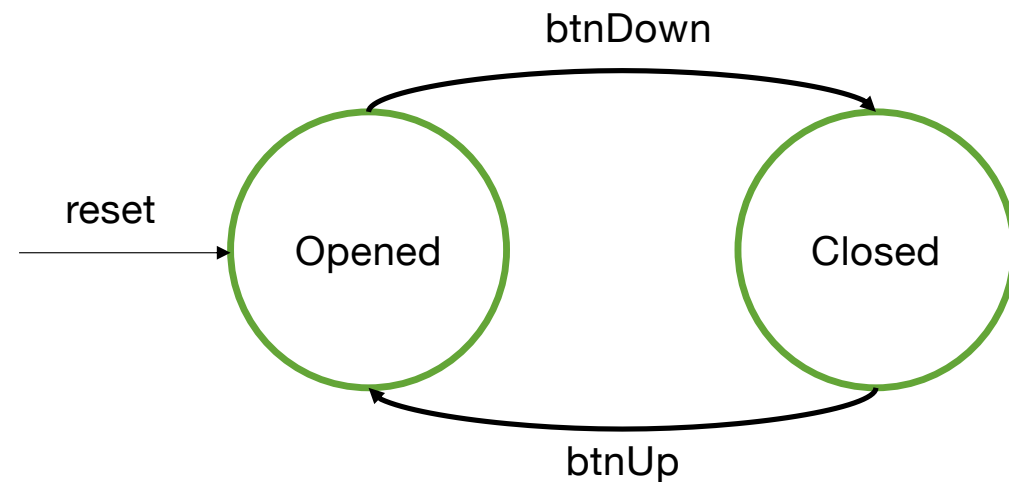


■ State determination

Opened
Closed

■ Transition condition

Button click (btnUp/btnDown)



Example 2

■ Garage Door Control

Remote control
Door Actuators

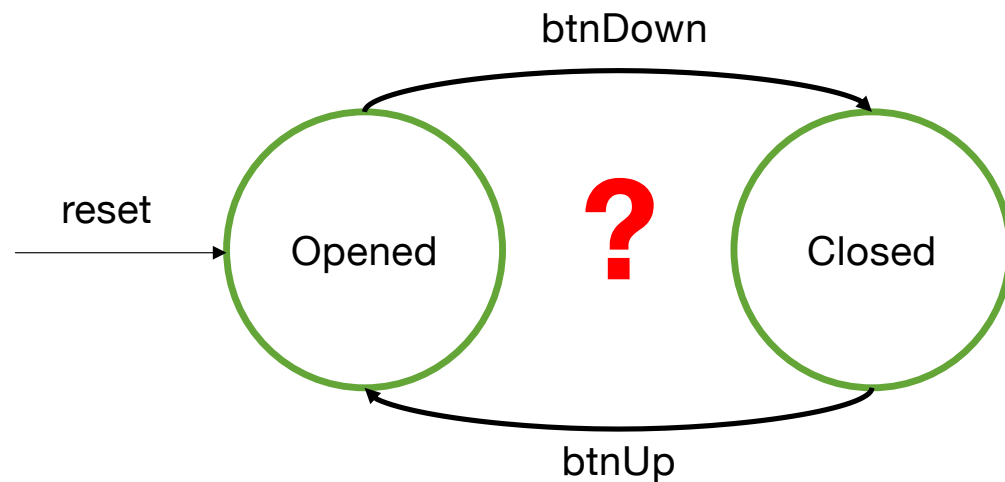


■ State determination

Opened
Closed

■ Transition condition

Button click (btnUp/btnDown)



Example 3

- **Garage Door Control**

Remote control
Door Actuators

- **State determination**

Opened
Closed
Opening
Closing

- **Transition condition**

Button click (btnUp/btnDown)
Sensors

Sensors



Example 3

■ Garage Door Control

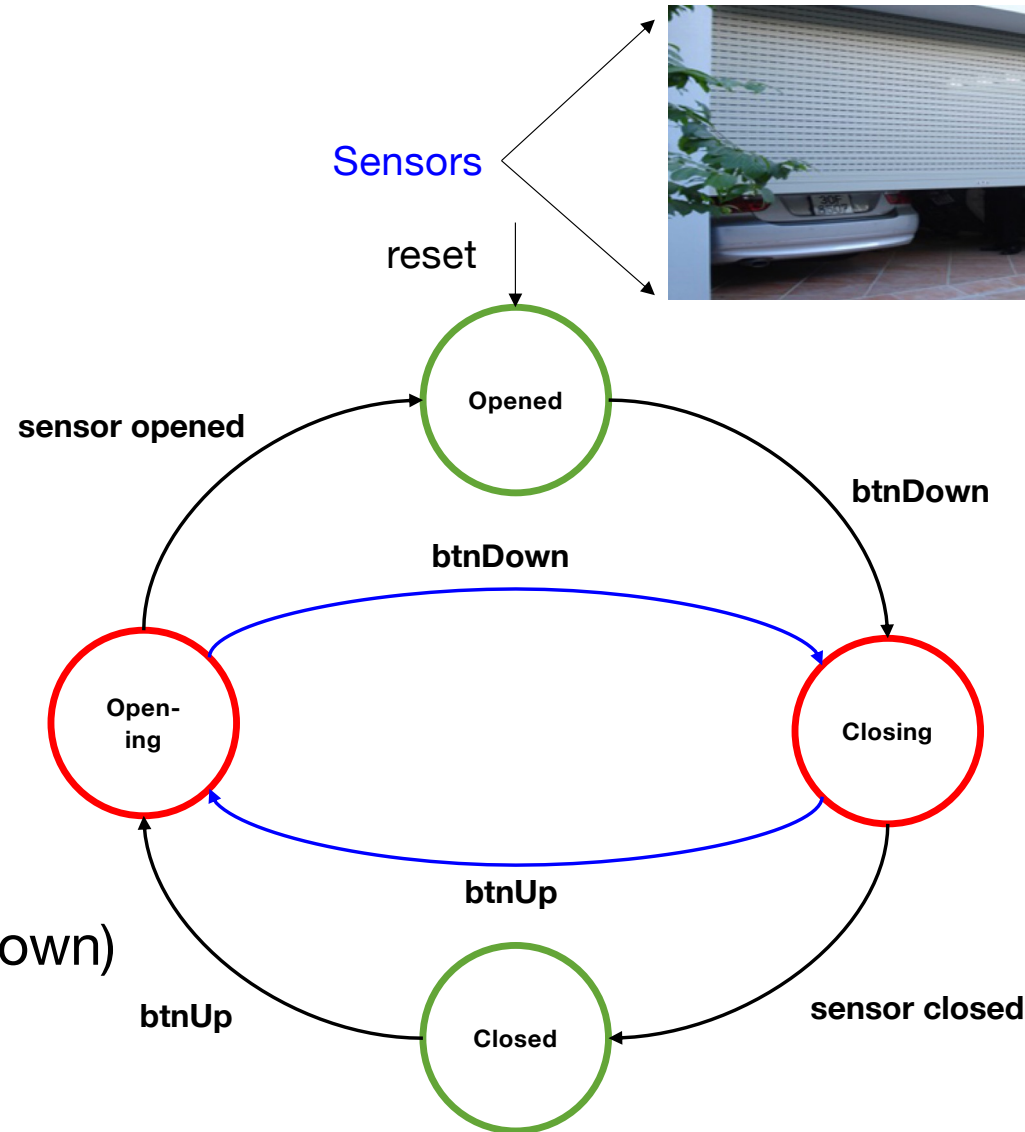
Remote control
Door Actuators

■ State determination

Opened
Closed
Opening
Closing

■ Transition condition

Button click (btnUp/btnDown)
Sensors



Example 3

■ Garage Door Control

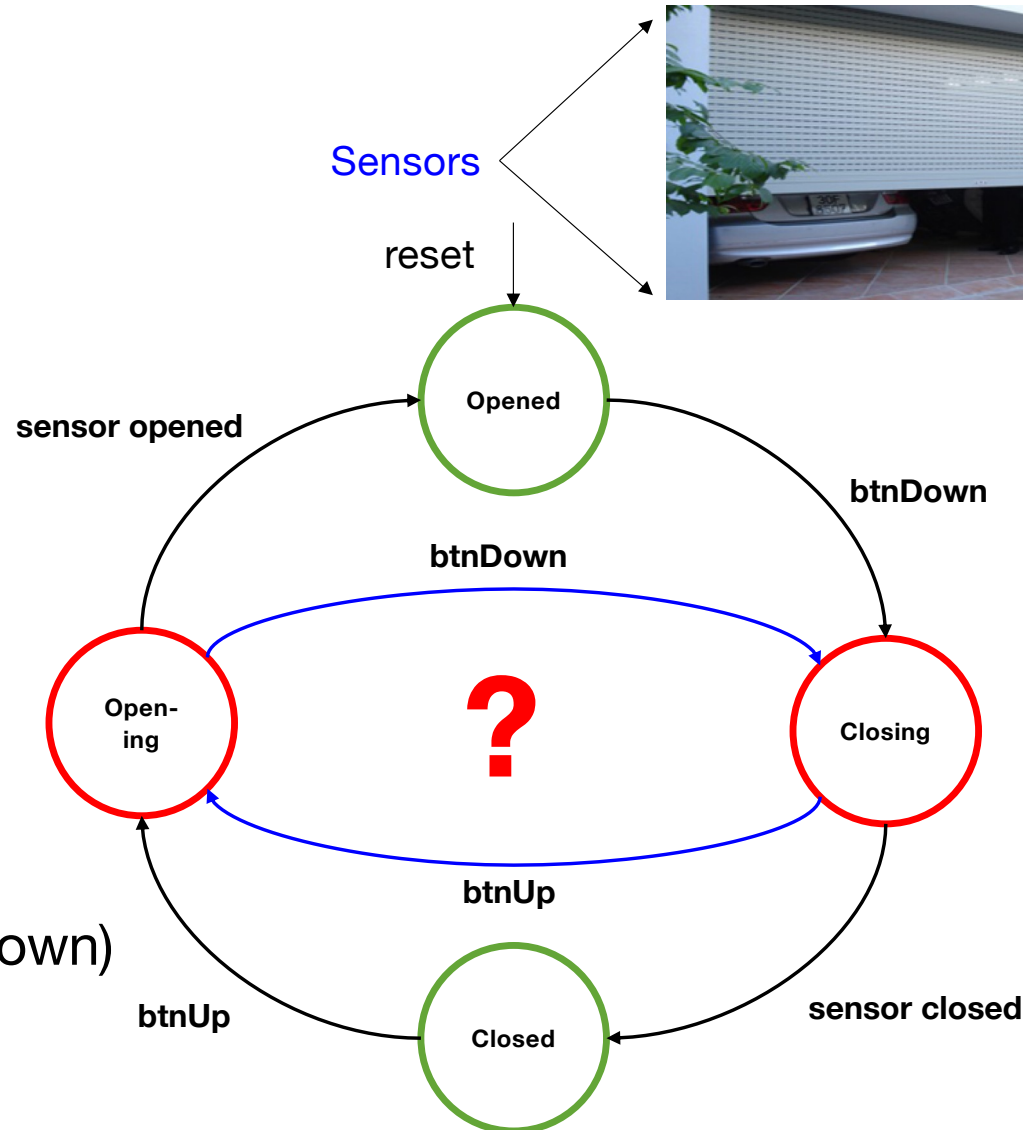
Remote control
Door Actuators

■ State determination

Opened
Closed
Opening
Closing

■ Transition condition

Button click (btnUp/btnDown)
Sensors



Example 4

■ Garage Door Control

Remote control
Door Actuators

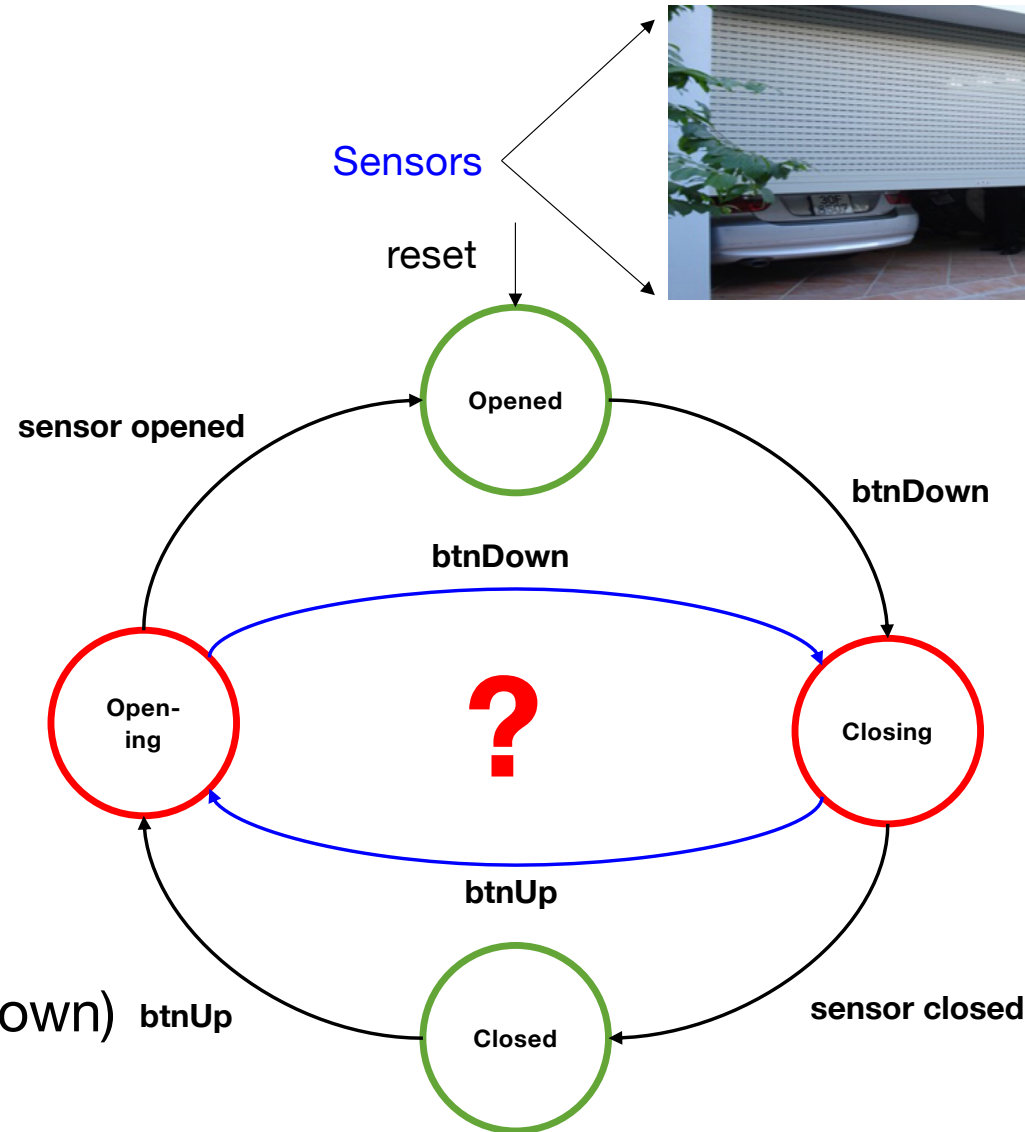
■ State determination

Opened
Closed
Opening
Closing
Half-Open

■ Transition condition

Button click (btnUp/btnDown)
Sensors

anhpham@hcmut.edu.vn



Example 4

■ Garage Door Control

Remote control
Door Actuators

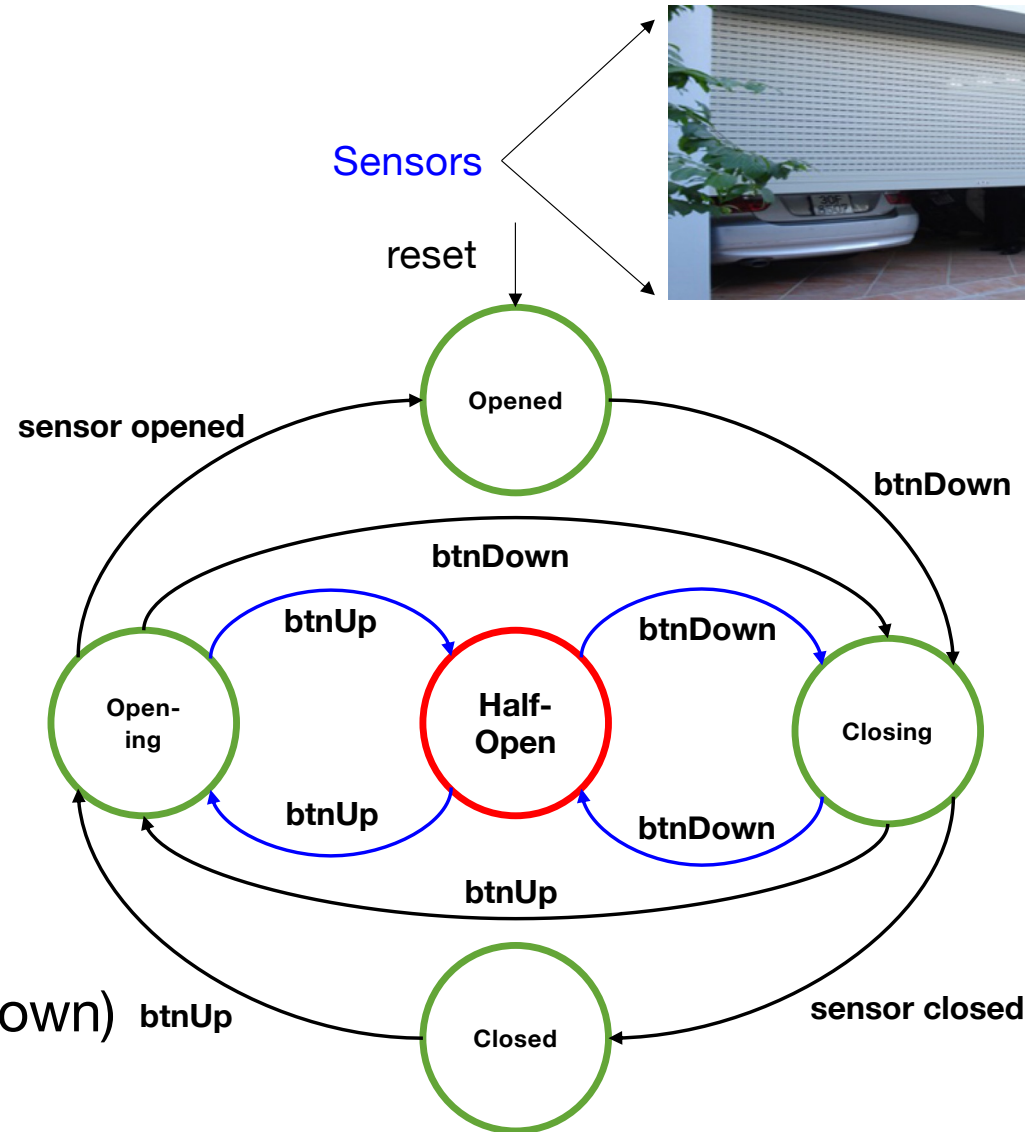
■ State determination

Opened
Closed
Opening
Closing
Half-Open

■ Transition condition

Button click (btnUp/btnDown)
Sensors

anhpham@hcmut.edu.vn



Common FSM Design Pitfalls

- **Missing transitions**

An event occurs in some state with no defined transition — exactly the gap the red '?' marks highlighted in the garage-door examples.

- **Unreachable states**

A state that no transition ever leads into — dead code in diagram form; usually a sign of a design or diagramming mistake.

- **Non-deterministic transitions**

Two transitions leaving the same state can both fire on the same event/condition at the same time — the system's next state becomes ambiguous.

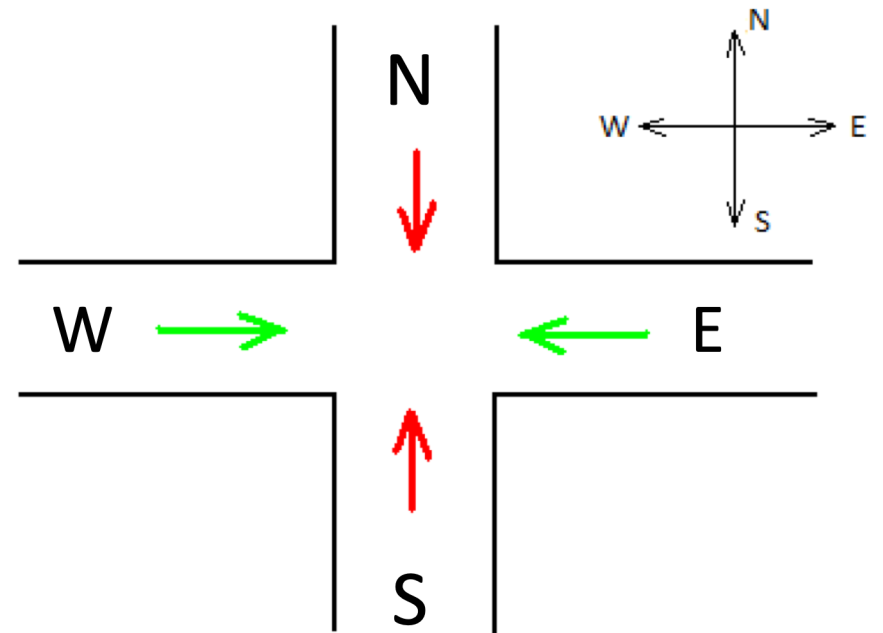
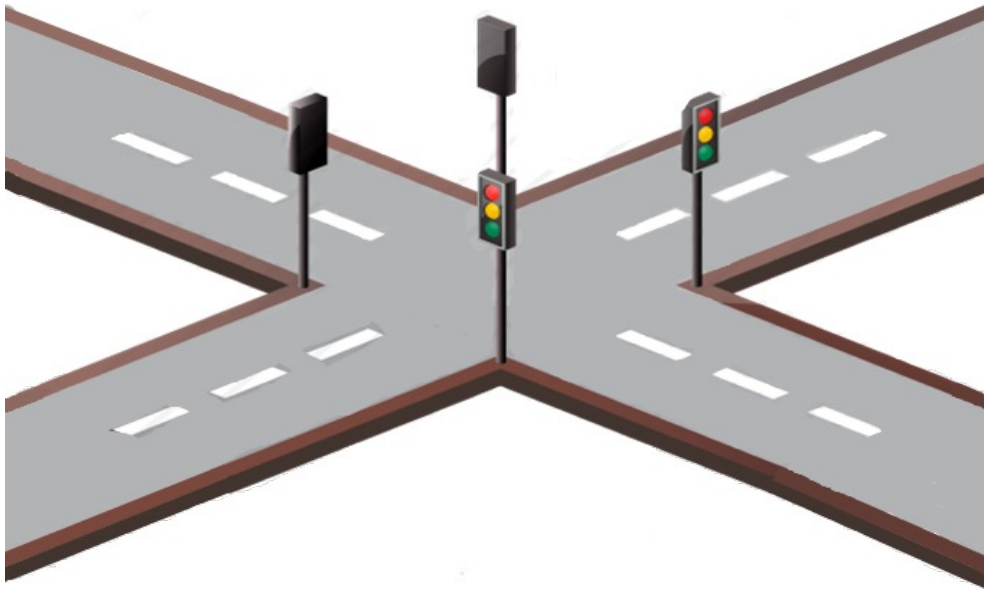
- **Ambiguous or unsafe direct transitions**

Exactly the Opening ➡ Closing problem we just resolved with the Half-Open state — a transition that is technically drawable but physically or logically questionable.

- **Best practice: for every state, explicitly ask 'what happens on every possible event here?' before considering the diagram finished.**



Exercise – Traffic Light 1



Exercise – Traffic Light 1

■ Traffic Light

Red, Yellow, Green
Timers

■ State determination

NSGo

- NS.Green (**on**), NS.Yellow (off), NS.Red (off)
- WE.Green (off), WE.Yellow (off), WE.Red (**on**)

NSToStop

- NS.Green (off), NS.Yellow (**on**), NS.Red (off)
- WE.Green (off), WE.Yellow (off), WE.Red (**on**)

WEGo

- WE.Green (**on**), WE.Yellow (off), WE.Red (off)
- NS.Green (off), NS.Yellow (off), NS.Red (**on**)

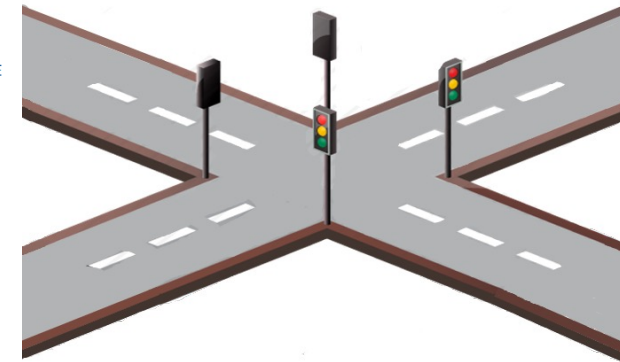
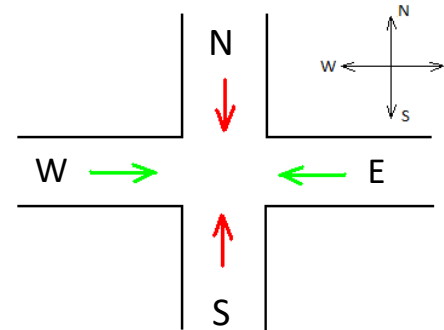
WEToStop

- WE.Green (off), WE.Yellow (**on**), WE.Red (off)
- NS.Green (off), NS.Yellow (off), NS.Red (**on**)

■ Transition condition

Timers

anhpham@hcmut.edu.vn



Exercise – Traffic Light 1

■ Traffic Light

Red, Yellow, Green
Timers

■ State determination

NSGo

- NS.Green (on), NS.Yellow (off), NS.Red (off)
- WE.Green (off), WE.Yellow (off), WE.Red (on)

NSToStop

- NS.Green (off), NS.Yellow (on), NS.Red (off)
- WE.Green (off), WE.Yellow (off), WE.Red (on)

WEGo

- WE.Green (on), WE.Yellow (off), WE.Red (off)
- NS.Green (off), NS.Yellow (off), NS.Red (on)

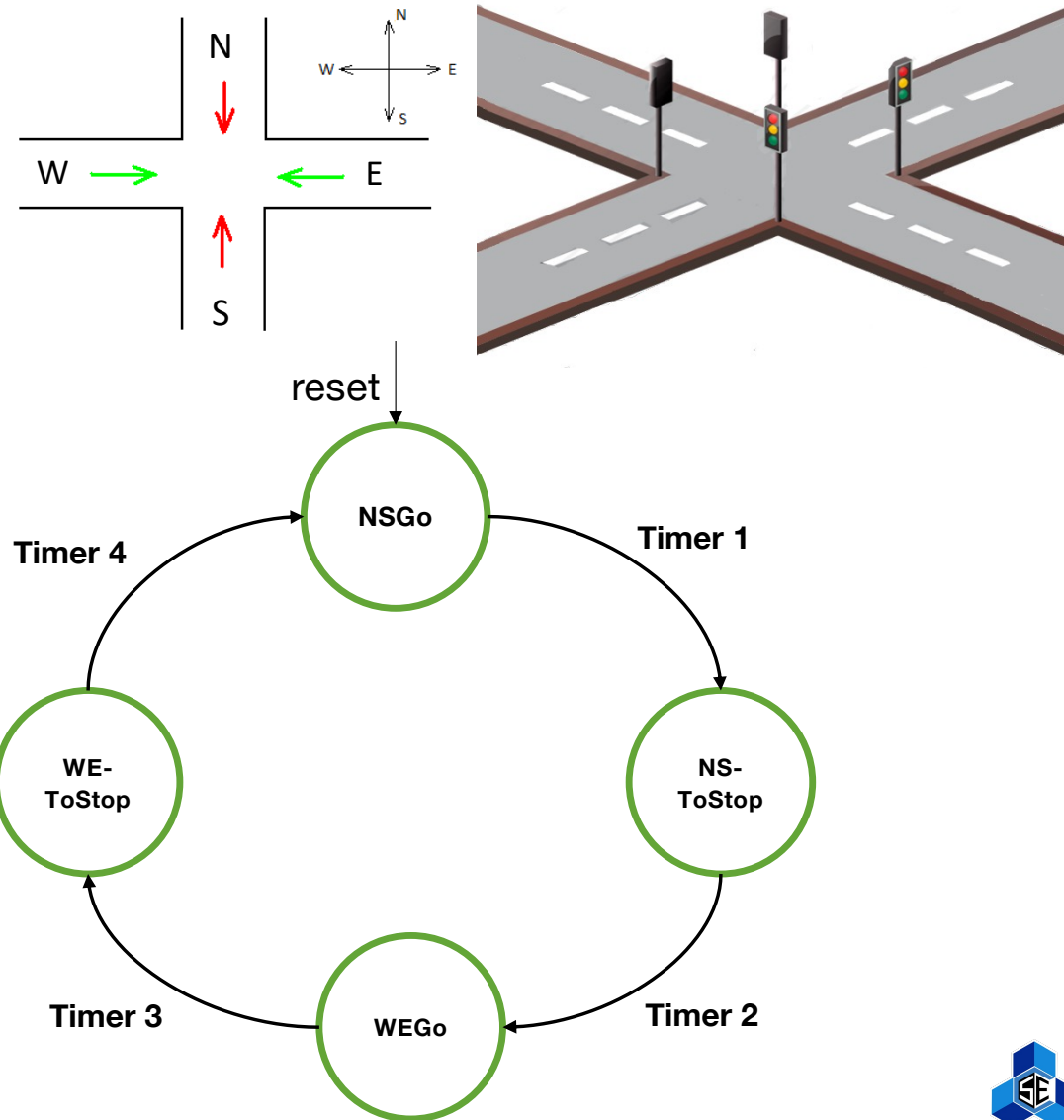
WEToStop

- WE.Green (off), WE.Yellow (on), WE.Red (off)
- NS.Green (off), NS.Yellow (off), NS.Red (on)

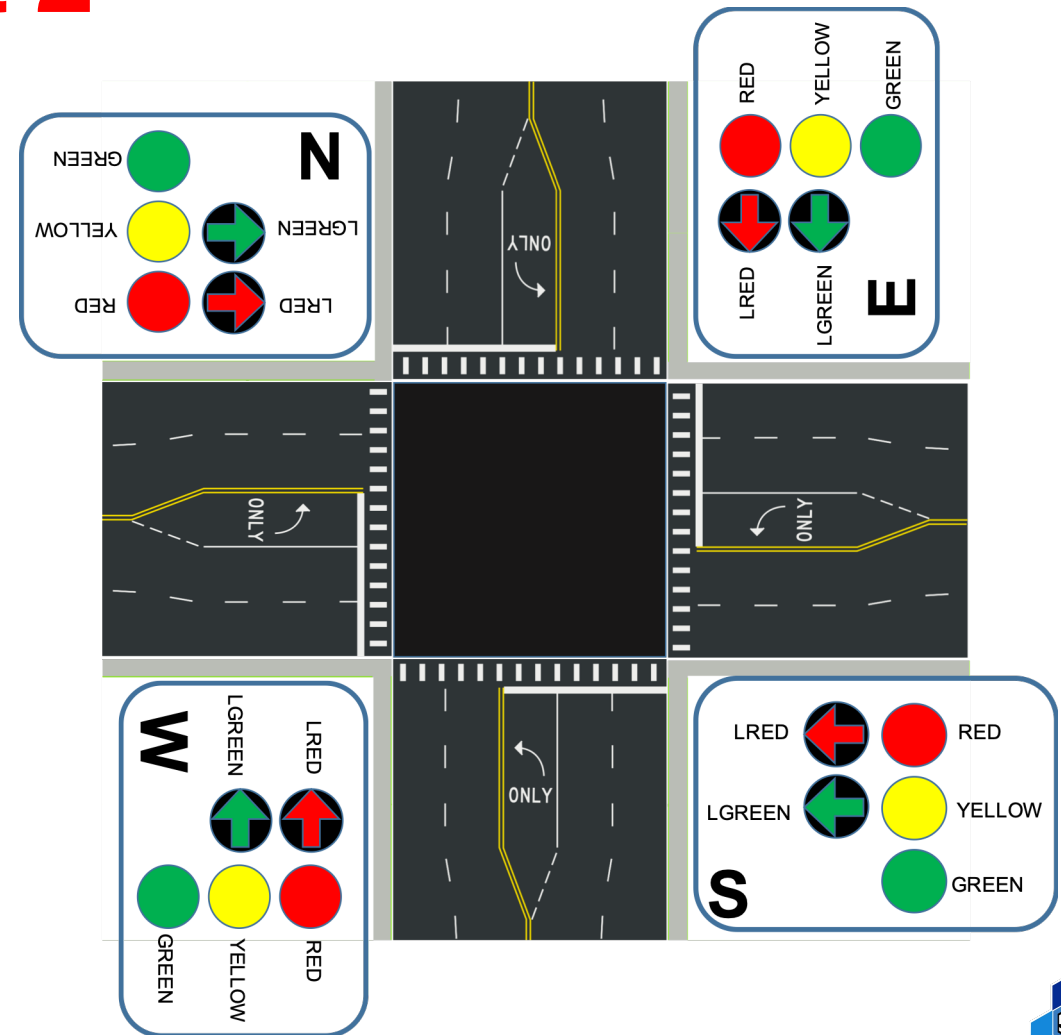
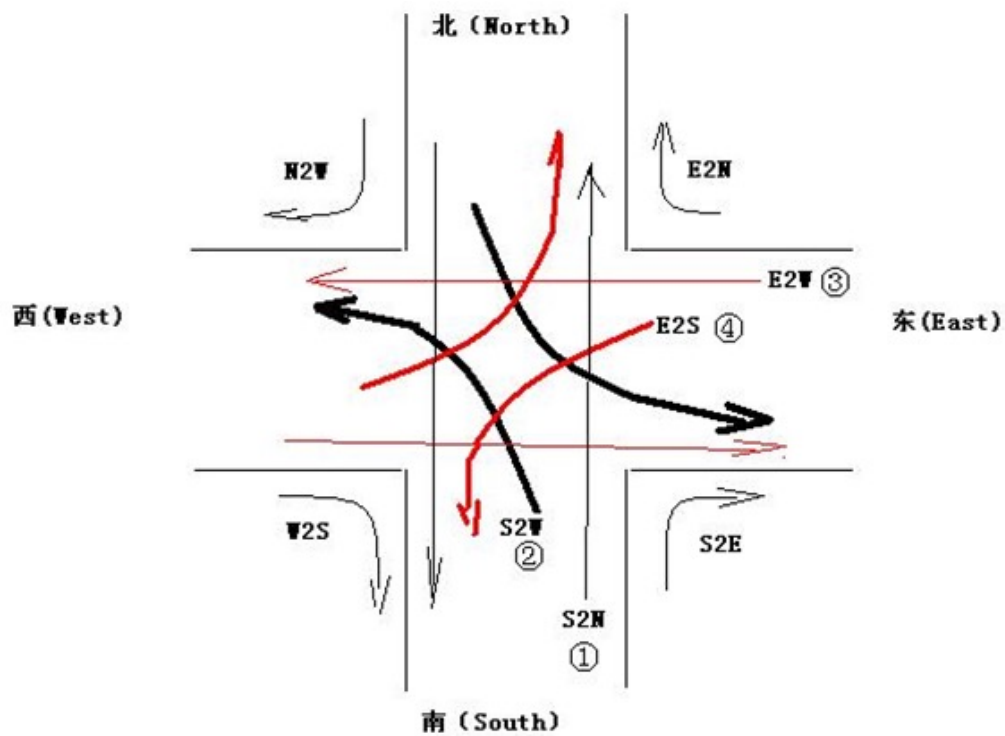
■ Transition condition

Timers

anhpham@hcmut.edu.vn



Exercise – Traffic Light 2



Hierarchical & Extended State Machines

■ Extended state machines

Add ordinary variables (not just states) to the model — e.g., a coin-total counter or a timer value — used in guard conditions alongside events.

The traffic light's Timer 1-4 are a simple example: a hidden countdown variable, not a separate state, drives each transition.

■ Hierarchical state machines (HSM / UML statecharts)

Allow a state to contain nested sub-states, so common transitions can be defined once at the parent level instead of being repeated in every child state.

Example: an 'Error' super-state could contain sub-states for different error types, all sharing one common 'reset button → back to Standby' transition defined only once.

■ Why it matters for embedded systems

Real products (like the washing-machine assignment) often have far more than 5 states — hierarchy keeps the diagram (and the code) manageable as complexity grows.



From Diagram to Code – Implementation Patterns

■ Switch-case on state

The simplest pattern: a switch statement on the current-state variable, with each case checking its own events/guards and updating the state variable.

Easy to write and read for a handful of states; can get unwieldy once a system has many states and shared transitions.

■ State-transition table (lookup table)

Store transitions as data — a table of (current state, event) → (next state, action) — and look them up at runtime instead of hard-coding branches.

Makes the transition logic easy to review against the diagram directly, since the table can mirror the diagram almost row for row.

■ State (design) pattern / function-pointer table

Represent each state as its own function (or object); a pointer/reference to 'the current state's handler' is called each cycle — common in C using arrays of function pointers, or in C++/OOP using polymorphism.



Testing State Machines

- **State coverage**

Verify that every state in the diagram is actually reachable and has been visited at least once during testing.

- **Transition coverage**

A stronger criterion: verify that every transition (every arrow) has been exercised at least once — not just every state.

A test suite can achieve 100% state coverage while completely missing a broken transition — always aim for transition coverage where feasible.

- **Connecting back to Lecture 3's V-Model**

State machine testing is a natural fit for unit testing on the host (simulate events, assert on state/output) before ever touching real target hardware.

The design pitfalls from the earlier slide (missing transitions, non-determinism) are exactly the kinds of bugs transition-coverage testing is designed to catch.



Flow-chart

- **To represent an algorithm for a specific function.**

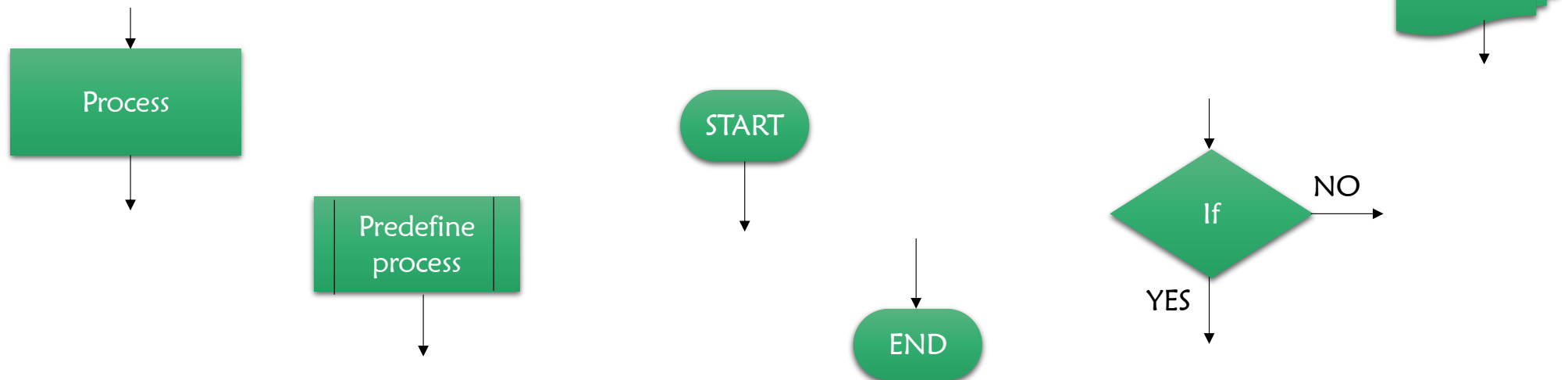
Use a rectangle for **a process**

Use a rounded rectangle for **a terminator** (START, END)

Use a diamond shape for **a decision**

Use a parallelogram for **data**

Use a rectangle with two vertical lines for **predefine process**



Flowchart vs. State Machine — When to Use Which

- **Use a flowchart when...**

You are describing a linear or branching computation that runs to completion — e.g., reading a sensor and deciding which LED to light.

There is no need to 'remember' being in a particular condition between separate triggering events.

- **Use a state machine when...**

The system must react differently to the same event depending on what has happened before (i.e., it needs memory of its current condition).

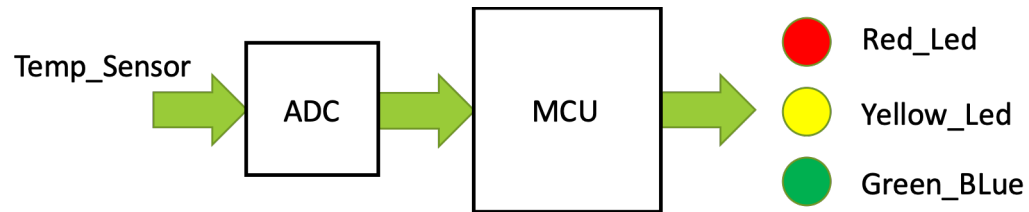
The system is reactive and long-running — it waits for events indefinitely rather than running once and finishing.

- **The temperature-LED example we just saw**

Both diagrams describe the exact same logic. As a one-shot decision made fresh from a new sensor reading each cycle, it is naturally a flowchart; when embedded inside a larger reactive system with more states and history, it is more natural to express as a state machine.

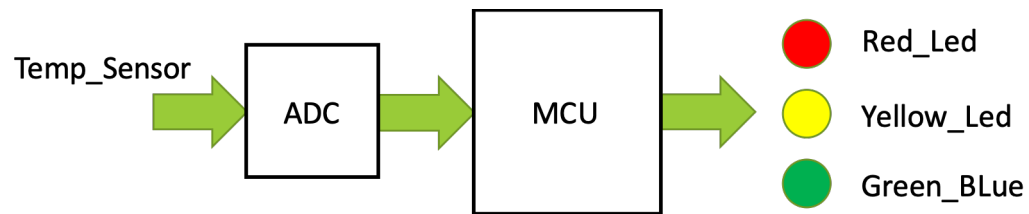


Flow-chart - Example

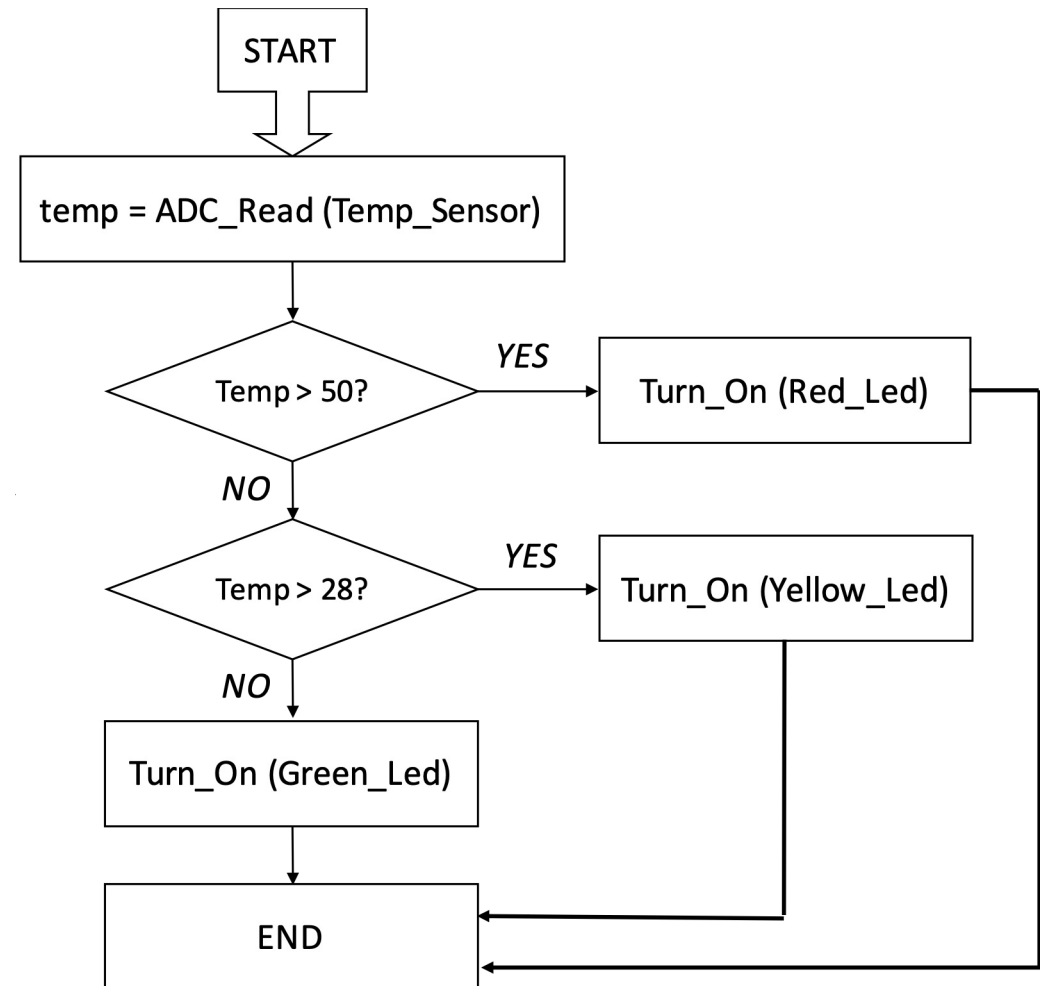


Notes: At a moment, only one Led is on while the two others are off

Flow-chart - Example



Notes: At a moment, only one Led is on while the two others are off



State Machine and Flow-Chart

State determination

GREEN_ON

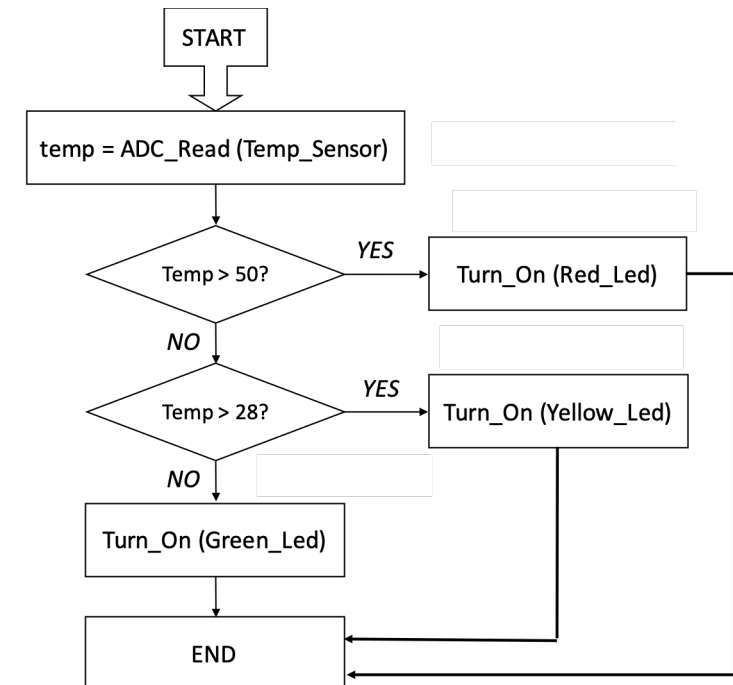
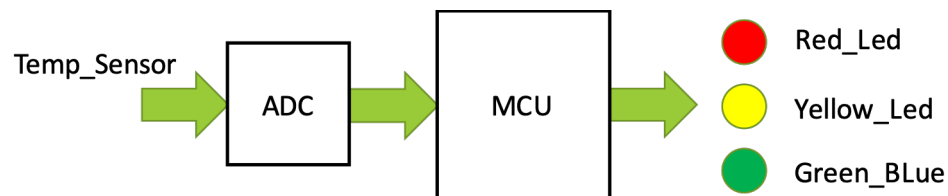
- Green_Led (on)
- Yellow_Led (off)
- Red_Led (off)

RED_ON

- Green_Led (off)
- Yellow_Led (off)
- Red_Led (on)

YELLOW_ON

- Green_Led (off),
- Yellow_Led (on),
- Red_Led (off)



Transition conditions

- T1: Temp ≤ 28
- T2: 28 < Temp ≤ 50
- T3: Temp > 50

State Machine and Flow-Chart

State determination

GREEN_ON

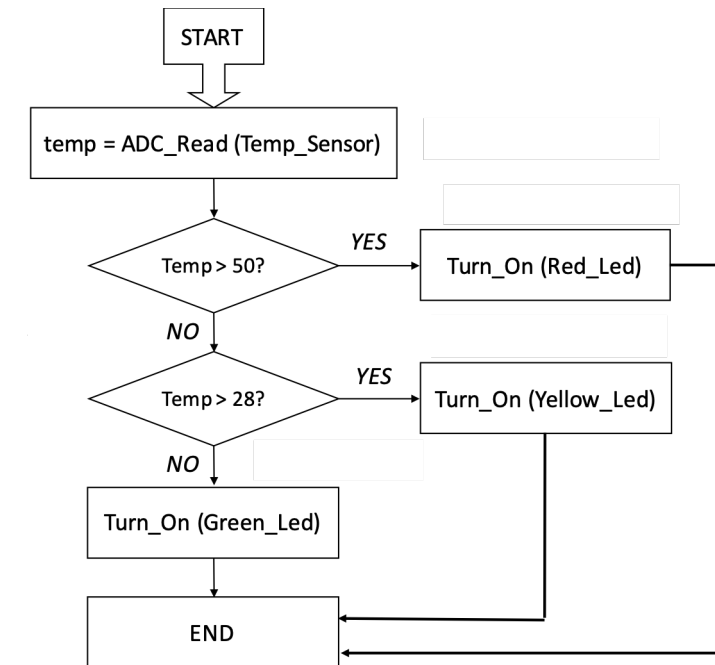
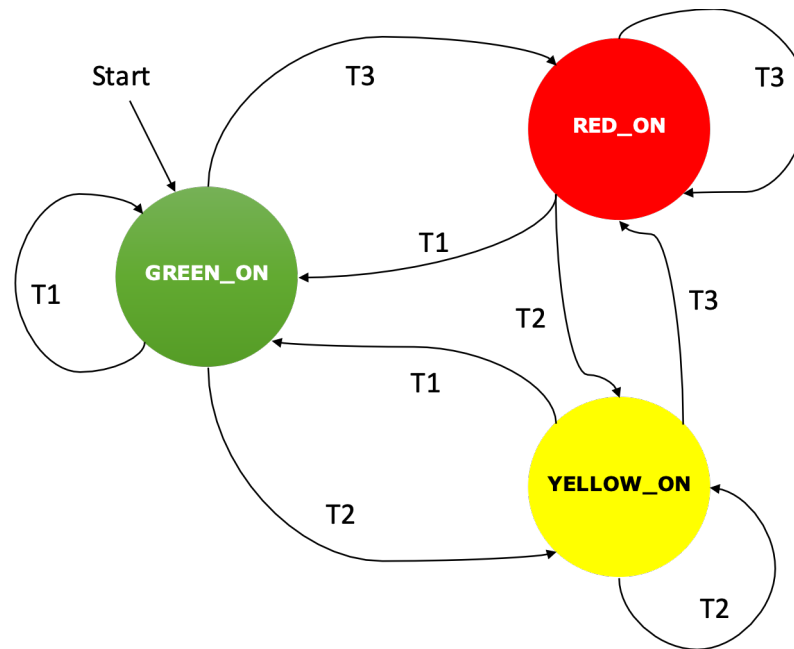
- Green_Led (**on**)
- Yellow_Led (off)
- Red_Led (off)

RED_ON

- Green_Led (off)
- Yellow_Led (off)
- Red_Led (**on**)

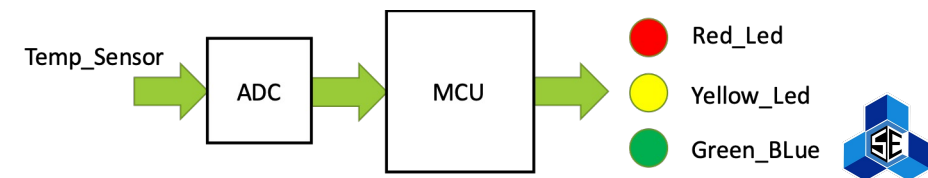
YELLOW_ON

- Green_Led (off),
- Yellow_Led (**on**),
- Red_Led (off)



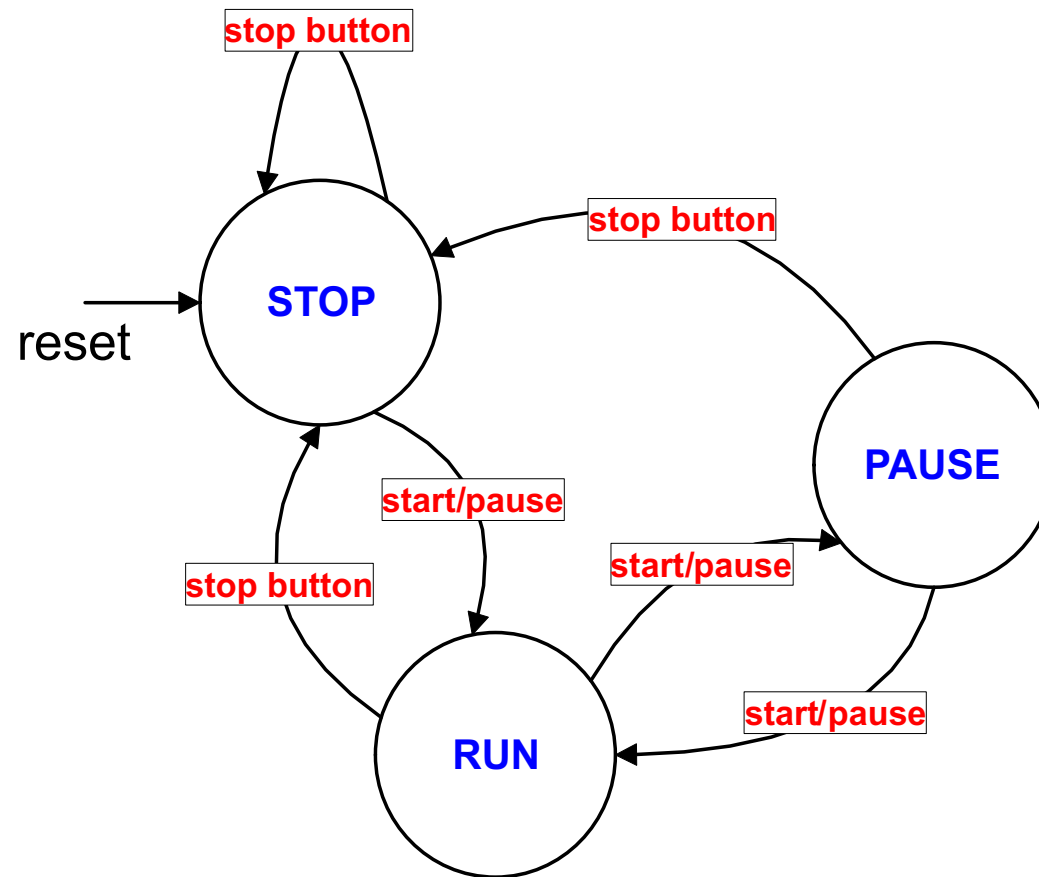
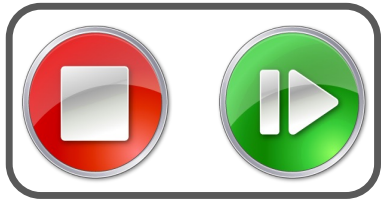
Transition conditions

- T1: $\text{Temp} \leq 28$
- T2: $28 < \text{Temp} \leq 50$
- T3: $\text{Temp} > 50$



Flow-chart - Exercise

- Represent the system described by the state machine in forms of flow-chart



Summary — Key Takeaways

- A state machine models how a system's behavior changes over time in response to events — built from states, transitions, events, guard conditions, and outputs.
- Moore machines (output depends on state only) are simpler and more stable; Mealy machines (output depends on state and input) can react faster but need more care.
- Good state-machine design is iterative: start simple, stress-test with 'what if' questions, and add states only to resolve genuine ambiguity — exactly what the garage-door examples demonstrated.
- Extended and hierarchical state machines scale these ideas up to real products with many states and shared transitions.
- A flowchart models a step-by-step algorithm; use it for one-shot computations, and a state machine for long-running reactive behavior — the two tools are complementary, not competing.
- Implementation matters: switch-case, transition tables, and the state pattern are the three standard ways to turn a diagram into working code, and transition coverage is the standard for testing it.



Assignment

- Design and implement the control unit of a washing machine that is described as follows:

The machine can be controlled by three buttons (STOP, RUN, PAUSE).

The machine has two LEDs including Red LED (RLED) and Blue one (BLED).

- RLED is on when machine is on standby (i.e., it is available to serve)
- RLED is blinking if the machine has errors
- BLED is on when the machine is ready to execute
- BLED is blinking when the machine is running (do washing)

The washing machine is only able to execute once at least 50-cents is added.

The machine only accepts 10-cent, 20-cent, 50-cent coins.

When there is enough money (BLED is on), the machine will execute if the RUN button is pressed. Then, a **30-mins timer will be activated** and the money will be clear without returning the redundancies (if any).

When the machine is running (BLED is blinking), it will be paused as the PAUSE button is pressed but **the timer is still counting down**. The machine will re-execute when the RUN button is pressed again.

The executing machine will be automatically terminated after 30 minutes (washing cycle) or when the STOP button is **pressed twice** (force to stop).

